

ftile_eth_400g_model.vhd

Line-by-line walkthrough, design rationale and review — Agilix 7 F-Tile Ethernet Hard IP, 400GE-8, MAC segmented client (behavioural model, simulation only)

How to read this document

Every section follows the file top to bottom. Each one shows the exact source lines (with their line numbers), then explains **what the code does**, **why it was written that way**, and **what is wrong or missing**. Line numbers refer to the file exactly as shipped in [eth400g_vhdl.zip](#).

The review findings are stated first, because several of them change how much the README's results table can be trusted.

Findings first

#	Finding	Where	Impact
1	Error verdict and random length are drawn once per beat, not once per frame. Two frames ending in the same beat are always both good or both bad.	rx_gen, lines 411-417, 446-447	The 'good frame shares a beat with a bad one' case that the drop stage relies on is never generated. The 'exact drop' result does not cover it.
2	The 8-bit LFSR (taps 7,5) has period 30, not 255. ERR_RATE=64 gives ~33% errors, not ~25%. ERR_RATE=255 is not 'all'. Stall pattern repeats every 30 cycles.	rng 334-335, tx_ready_gen 537-538	Poor, periodic coverage of error positions and stall/pipeline phase.
3	AVG_IPG is dead code. ipg_cnt is reset to 0 and decremented but never loaded.	489-501	The generic does nothing. If it were loaded, it would discard a whole beat of data instead of inserting gap.
4	GAP_SEGS=0 produces a stream with no inframe 1->0 transition between frames. By the file's own EOP definition (line 34) the frame boundary is invisible.	rx_gen 474, comment 343-347	The 'gap=0' results row describes an unframeable stream. Also: gap=1 at 64 B is ~738 Mpps, above the ~595 Mpps physical 400GE limit.
5	TX and RX reset acks are the same signal; per-direction resets are never acknowledged; link status never drops mid-traffic.	308-320	Code that depends on ack order, or that mishandles link loss, passes here and fails on hardware.
6	TX freeze check compares only data and inframe; no TX frame-content check; TX_MAX_FRAME never read.	tx_check 588-601	eop_empty/error/skip_crc changes during pause, illegal framing and oversize TX frames (your +64 B encapsulation) are all undetected.
7	ENFORCE_MAX flags but does not truncate; oversize is not flagged when ENFORCE_MAX=false; undersize never flagged; STRIP_CRC_PAD branch is equivalent to STRIP_CRC.	250-262, 452-464	Features that appear modelled are not.

None of these affect synthesis — this file never goes into Quartus. They affect how much confidence a clean simulation run should give you.

1. Header comment block (lines 1-54)

```
1  =====
2  -- ftile_eth_400g_model.vhd
3  --
4  -- BEHAVIOURAL MODEL of the Agilex 7 F-Tile Ethernet Hard IP, 400GE-8,
5  -- MAC SEGMENTED client interface.
6  --
7  -- SIMULATION ONLY. Delete this and instantiate the licensed IP when
8  -- available; the datapath connects to the same signal names.
9  --
10 -- Reference: F-Tile Ethernet Hard IP User Guide, doc 683023 (2026.08.10)
11 --
12 --=====
13 -- IP FEATURES MODELLED (and the UG section each comes from)
14 --=====
15 -- sec 5 Clocks 415.0390625 MHz MAC datapath clock
16 -- sec 6.2 Reset sequence 12-step bring-up, 6 reset/ack signals
17 -- sec 7.1 Status tx_lanes_stable, rx_pcs_ready, block_lock,
18 -- am_lock, pcs_fully_aligned, local/remote fault
19 -- sec 7.4 TX segmented fixed-latency PAUSE protocol, tight packing,
20 -- skip_crc, tx error
21 -- sec 7.5 RX segmented no backpressure, per-segment inframe /
22 -- eop_empty / fcs_error / error / status
23 -- MAC opts TX/RX max frame size (65..65535), enforce max frame size,
24 -- bytes to remove from RX frames, average IPG, VLAN detection
25 --
26 --=====
```

What it is. A statement of scope. The model claims to cover five areas of the IP, each tied to a User Guide section: clocks (sec 5), reset sequence (sec 6.2), status (sec 7.1), TX segmented protocol (sec 7.4) and RX segmented protocol (sec 7.5), plus the MAC option parameters.

Why it is written this way. Tying every modelled feature to a UG section makes the model auditable: when the real IP disagrees, you know which paragraph to reread. It also documents what is **not** modelled by omission — PMA, PCS, RS-FEC, AN/LT, PAUSE/PFC, statistics and PTP are absent.

Why a model at all. The real F-Tile Hard IP needs a license, Quartus Prime Pro with Agilex 7 support, and its simulation model is large and slow. A behavioural stand-in with identical port names lets you build and debug `eth400g_loopback` today, then swap in the real IP without editing the datapath.

```
27 -- KEY INTERFACE SEMANTICS
28 --=====
29 -- The MAC segmented interface does NOT use sop/eop strobes. It uses
30 -- INFRAME + EOP_EMPTY:
31 --
32 -- *_inframe(s) '1' = segment s is inside a frame
33 -- SOP = inframe 0 -> 1 between consecutive segments
34 -- EOP = inframe 1 -> 0 between consecutive segments
35 -- *_eop_empty(s) unused bytes in segment s, valid only at EOP
36 --
37 -- UG sec 7.5: "Packets may start on any 8-byte segment. For multisegmented
38 -- interfaces, a new packet may start and the previous packet end are within
39 -- the same cycle."
40 --
41 -- At 400G a beat is 128 bytes and the minimum frame is 64 bytes, so TWO
42 -- complete frames land in ONE cycle routinely. Not a corner case.
43 --
44 -- TX IS A FIXED-LATENCY PAUSE INTERFACE, NOT READY/VALID (UG sec 7.4):
45 -- "i_tx_mac_valid asserts only when o_tx_mac_ready is asserted, even
46 -- though there is no packet to send"
47 -- "...can be spaced by a fixed latency between 1 to 7 clock cycles"
48 -- "When i_tx_mac_valid deasserts, [all TX signals] must be paused for as
49 -- many cycles as o_tx_mac_ready is deasserted"
50 --
51 -- NOTE: the 'Ready latency' IP PARAMETER (range 0-3) has NO EFFECT in MAC
52 -- segmented mode. The 1-7 cycle spacing is a latency YOU choose and hold
53 -- consistently, not something configured in the IP.
54 --=====
```

The key interface semantics. This block is the most important comment in the file, because the rest of the design depends on it:

- There are **no sop/eop strobes**. Framing is carried entirely by `inframe(s)` per segment. A start-of-packet is a 0->1 transition of inframe between consecutive segments; an end-of-packet is a 1->0 transition.

- `eop_empty(s)` says how many of the 8 bytes in the EOP segment are unused. It is only meaningful at EOP.
- A beat is 128 bytes (16 segments x 8 bytes) and a minimum frame is 64 bytes, so **two complete frames in one clock cycle is normal traffic**, not a corner case. Any design that assumes 'one frame per beat' or 'one EOP per beat' is wrong at 400G.
- **TX is not ready/valid.** `valid` must equal `ready` delayed by a fixed latency (1-7 cycles) that *you* choose and hold. When valid is low, every TX signal must be frozen.

Unverified claim: the note on lines 51-53 that the 'Ready latency' IP parameter has no effect in MAC segmented mode is attributed to the UG. I cannot confirm it from here. Check it against your IP version before relying on it.

2. Library clauses (lines 55-58)

```
55 library ieee;
56 use ieee.std_logic_1164.all;
57 use ieee.numeric_std.all;
58 use work.eth400g_pkg.all;
```

- `std_logic_1164` gives `std_logic` and `std_logic_vector`, the resolved 9-value types used for all ports.
- `numeric_std` gives `unsigned` and arithmetic on it. It is the IEEE-standard package; the non-standard `std_logic_arith/std_logic_unsigned` are deliberately not used, which avoids signed/unsigned ambiguity and keeps the code portable across GHDL, Questa and Quartus.
- `work.eth400g_pkg` supplies the geometry constants (`DATA_W=1024`, `NUM_SEG=16`, `SEG_B=8`, `EMPTY_W=3`, `ERR_W=2`, `STAT_W=3`), the error/status codes, frame limits and the `rx_strip_t` type. Keeping these in one package means the model, datapath and testbench can never disagree on bus widths.

3. Entity generics (lines 60-80)

```
60 entity ftile_eth_400g_model is
61     generic (
62         -- TX pause spacing you have chosen to implement. Legal 1..7.
63         READY_LATENCY : natural range 1 to 7 := 3;
64
65         -- UG MAC options. Jumbo is a PARAMETER: range 65..65535.
66         RX_MAX_FRAME   : natural := DEF_MAX_FRAME;
67         TX_MAX_FRAME   : natural := DEF_MAX_FRAME;
68         ENFORCE_MAX    : boolean := false;    -- truncate oversized, flag error
69         RX_STRIP       : rx_strip_t := STRIP_CRC;
70
71         -- 'Average inter-packet gap': 1, 8, 10 or 12. Default 12 is standard.
72         -- UG warns values below 12 mean "the IP core no longer complies with
73         -- the Ethernet standard" - it buys throughput at small frame sizes.
74         AVG_IPG        : natural range 1 to 12 := 12;
75
76         -- Traffic generation (NOT IP features - model control only)
77         MIN_LEN        : natural := MIN_ETH_FRAME;
78         MAX_LEN        : natural := DEF_MAX_FRAME;
79         SEED           : natural := 12345
80     );
```

Generic	Default	Meaning	Why chosen / notes
READY_LATENCY	3	Cycles between o_tx_mac_ready and the i_tx_mac_valid it permits. Range-constrained 1..7.	The range constraint turns an illegal value into an elaboration error instead of a silent protocol mismatch. Must match the latency your datapath implements.
RX_MAX_FRAME	1518	Largest RX frame the MAC accepts (UG range 65..65535).	Makes jumbo a parameter. Only used when ENFORCE_MAX is true (line 452).
TX_MAX_FRAME	1518	Largest TX frame.	Never read anywhere in the file. Your +64 B tunnel overhead makes this the check you most need.
ENFORCE_MAX	false	Truncate oversize RX frames and flag them.	Code flags but does not truncate (section 12.6).
RX_STRIP	STRIP_CRC	What the MAC removes before handing RX to you: nothing, CRC, or CRC+pad.	Decides what your encryptor sees. With CRC stripped, a 64 B wire frame arrives as 60 B.
AVG_IPG	12	Average inter-packet gap setting (1/8/10/12).	Has no effect (section 12.8). The range 1..12 also admits illegal values 2-7, 9, 11.
MIN_LEN / MAX_LEN	64 / 1518	Random frame length bounds for the generator.	Model control only. Not IP features.
SEED	12345	Initial value of the 32-bit length LFSR.	Different seeds give different length sequences; reproducible runs.

The split between 'UG MAC options' and 'traffic generation (NOT IP features)' is deliberate: when you replace the model with the real IP, the first group becomes IP parameter-editor settings and the second group simply disappears.

4. Entity ports (lines 81-152)

4.1 Clocks (lines 82-89)

```
82     -----
83     -- CLOCKS (UG sec 5)
84     -- In MAC segmented mode i_clk_tx and i_clk_rx MUST come from
85     -- o_clk_pll. 'Enable asynchronous adapter clocks' is available only
86     -- for MAC Avalon ST, so these cannot be independent here.
87     -----
88     i_clk_tx      : in std_logic;
89     i_clk_rx      : in std_logic;
```

Two clock ports exist because the real IP has two, and the whole point of the model is name compatibility. In MAC segmented mode they must be the same 415.0390625 MHz clock from `o_clk_pll`; asynchronous adapter clocks are only offered for the Avalon-ST client (per the comment — verify against your IP version).

Why 415.04 MHz. 1024 bits x 415.0390625 MHz = 425 Gb/s of interface capacity. That is ~6% more than 400 Gb/s; the surplus shows up as idle segments. It is not 390.625 MHz (that is `o_clk_tx_div`, used

for PTP/TOD).

Consequence for an I-Series device. If the single-clock rule holds, all fabric logic facing the IP runs at 415 MHz regardless of device Fmax. A 650 MHz-capable device only helps if you move the crypto core into a faster clock domain behind CDC FIFOs.

4.2 Resets (lines 91-100)

```
91      -----
92      -- RESETS   (UG sec 6.2)
93      -----
94      i_rst_n      : in  std_logic;   -- full IP reset
95      i_tx_rst_n   : in  std_logic;   -- TX datapath reset
96      i_rx_rst_n   : in  std_logic;   -- RX datapath reset
97
98      o_rst_ack_n   : out std_logic;
99      o_tx_rst_ack_n : out std_logic;
100     o_rx_rst_ack_n : out std_logic;
```

Port	Meaning
i_rst_n	Full IP reset, active low. Resets both directions and the whole tile datapath.
i_tx_rst_n / i_rx_rst_n	Per-direction datapath resets. Used for recovery of one direction without disturbing the other.
o_rst_ack_n	Acknowledge of i_rst_n. Its deassertion means 'out of full reset', which the UG says does not yet mean fully functional.
o_tx_rst_ack_n / o_rx_rst_ack_n	Per-direction acknowledges. Their relative order is not guaranteed by the UG.

4.3 Status (lines 102-112)

```
102     -----
103     -- STATUS   (UG sec 7.1)   all asynchronous on the real IP
104     -----
105     o_tx_lanes_stable : out std_logic;
106     o_rx_pcs_ready    : out std_logic;
107     o_rx_block_lock   : out std_logic;
108     o_rx_am_lock      : out std_logic;
109     o_rx_pcs_fully_aligned : out std_logic;
110     o_local_fault_status : out std_logic;
111     o_remote_fault_status : out std_logic;
112     o_rx_hi_ber       : out std_logic;
```

Port	Meaning on the real IP	What the model drives
o_tx_lanes_stable	TX lanes stable; TX datapath usable. Gate all TX on this.	s_lanes after ~24 cycles
o_rx_pcs_ready	RX PCS locked and aligned; RX data valid.	s_pcs_rdy after ~32 cycles
o_rx_block_lock	66b block lock on all PCS lanes.	= s_pcs_rdy
o_rx_am_lock	Alignment-marker lock.	= s_pcs_rdy
o_rx_pcs_fully_aligned	Lane deskew complete.	= s_pcs_rdy
o_local_fault_status	Local fault (RX not working).	= not s_pcs_rdy
o_remote_fault_status	Link partner reports a fault.	Always '0'
o_rx_hi_ber	High bit-error rate.	Always '0'

The comment 'all asynchronous on the real IP' matters: your design must synchronise every one of these before using it in the 415 MHz domain. The model drives them synchronously, so simulation cannot catch a missing synchroniser.

4.4 Model control (lines 114-121)

```
114     -----
115     -- MODEL CONTROL - NONE OF THESE EXIST ON THE REAL IP
116     -----
117     i_gen_enable     : in  std_logic;
118     i_force_len      : in  unsigned(15 downto 0); -- 0 = pseudo-random
119     i_gap_segs       : in  unsigned(2 downto 0);  -- idle segs between frames
120     i_err_rate       : in  unsigned(7 downto 0);  -- 0 = none, 255 = all
121     i_tx_stall_rate  : in  unsigned(7 downto 0);  -- ready deassert rate
```

Port	Effect
i_gen_enable	'1' lets the RX generator start new frames. A frame already in progress always finishes (line 387), so disabling never truncates.
i_force_len	Non-zero: every frame this many bytes. Zero: pseudo-random between MIN_LEN and MAX_LEN.
i_gap_segs	Idle segments inserted after each frame (0-7).
i_err_rate	Probability-like threshold for error injection, compared against an 8-bit LFSR.
i_tx_stall_rate	Threshold for deasserting o_tx_mac_ready; 0 = never stall.

None of these exist on the real IP. They are grouped and labelled so the testbench wiring that drives them is easy to delete.

4.5 RX client interface (lines 123-132)

```

123      -----
124      -- RX MAC SEGMENTED CLIENT  (IP -> user).  Takes NO backpressure.
125      -----
126      o_rx_mac_data      : out std_logic_vector(DATA_W-1 downto 0);
127      o_rx_mac_valid     : out std_logic;
128      o_rx_mac_inframe   : out std_logic_vector(NUM_SEG-1 downto 0);
129      o_rx_mac_eop_empty : out std_logic_vector(NUM_SEG*EMPTY_W-1 downto 0);
130      o_rx_mac_fcs_error : out std_logic_vector(NUM_SEG-1 downto 0);
131      o_rx_mac_error     : out std_logic_vector(NUM_SEG*ERR_W-1 downto 0);
132      o_rx_mac_status_data : out std_logic_vector(NUM_SEG*STAT_W-1 downto 0);

```

Port	Width	Meaning
o_rx_mac_data	1024	16 segments x 64 bits. In this model, frame byte k of segment s sits at bits (s*8+k)*8+7 downto (s*8+k)*8, so segment 0 and byte 0 are at the LSB end (line 431).
o_rx_mac_valid	1	Beat carries data. In this model '0' on beats with no inframe bits set.
o_rx_mac_inframe	16	Per-segment 'inside a frame' flag. Framing is derived from its transitions.
o_rx_mac_eop_empty	48	3 bits per segment: unused bytes in the EOP segment (0-7).
o_rx_mac_fcs_error	16	Per-segment CRC error, on the EOP segment.
o_rx_mac_error	32	2 bits per segment: 0 none, 1 malformed, 2 under/oversize, 3 payload length.
o_rx_mac_status_data	48	3 bits per segment: frame classification (VLAN, SVLAN, PFC, bcast/mcast, ...).

No ready input. RX takes no backpressure: the IP pushes and you must absorb every beat. That single fact dictates the downstream architecture — a FIFO that must drop when full, because there is nothing to stall.

Verify byte ordering against the UG. The model puts the first frame byte in the least significant byte of segment 0. If the real IP places byte 0 at the most significant end of each segment, every header field your encryptor parses or inserts will be byte-swapped, and the model will not show it.

4.6 TX client interface (lines 134-143)

```

134      -----
135      -- TX MAC SEGMENTED CLIENT  (user -> IP)
136      -----
137      i_tx_mac_data      : in  std_logic_vector(DATA_W-1 downto 0);
138      i_tx_mac_valid     : in  std_logic;
139      i_tx_mac_inframe   : in  std_logic_vector(NUM_SEG-1 downto 0);
140      i_tx_mac_eop_empty : in  std_logic_vector(NUM_SEG*EMPTY_W-1 downto 0);
141      i_tx_mac_error     : in  std_logic_vector(NUM_SEG-1 downto 0);
142      i_tx_mac_skip_crc  : in  std_logic_vector(NUM_SEG-1 downto 0);
143      o_tx_mac_ready     : out std_logic;

```

Mirror image of RX with two differences: **i_tx_mac_error** is 1 bit per segment (mark a frame bad so the MAC corrupts its FCS), and **i_tx_mac_skip_crc** lets you supply your own FCS instead of having the MAC append one. **o_tx_mac_ready** is the only signal flowing IP-to-user on TX.

4.7 Model observation (lines 145-152)

```

145      -----
146      -- MODEL OBSERVATION - not on the real IP
147      -----
148      o_model_rx_frames  : out unsigned(31 downto 0);
149      o_model_bad_frames : out unsigned(31 downto 0);
150      o_model_proto_viol : out unsigned(31 downto 0)
151  );
152  end entity ffile_eth_400g_model;

```

Counters exposed to the testbench: frames generated, frames marked bad, and TX protocol violations detected. The testbench compares these with what the datapath delivered to prove exact drop counts and zero violations.

5. Architecture declarations (lines 155-218)

5.1 Reset/status sequencing constants and signals (lines 157-185)

```
175     constant RST_ACK_CYC    : natural := 8;    -- i_rst_n high -> o_rst_ack_n low
176     constant LANE_CYC      : natural := 24;    -- -> tx_lanes_stable
177     constant PCS_CYC       : natural := 32;    -- -> rx_pcs_ready, alignment
178
179     signal rst_cnt         : natural range 0 to 63 := 0;
180     signal tx_cnt         : natural range 0 to 63 := 0;
181     signal rx_cnt         : natural range 0 to 63 := 0;
182
183     signal s_rst_ack_n    : std_logic := '0';
184     signal s_lanes       : std_logic := '0';
185     signal s_pcs_rdy     : std_logic := '0';
```

The three constants create an ordering in simulation: acknowledge after 8 cycles, TX lanes stable 24 cycles later, RX PCS ready 32 cycles after the ack. The absolute values are arbitrary — the real IP takes microseconds to milliseconds for PCS lock. What matters is that the datapath sees **reset released -> ack -> lanes stable -> PCS ready** and cannot shortcut it.

The counters are **natural range 0 to 63**, so synthesis-style sizing is 6 bits; the **:=** initialisers matter only for simulation start-up.

5.2 Pseudo-random sources (lines 187-192)

```
187     -----
188     -- Pseudo-random source
189     -----
190     signal lfsr      : unsigned(31 downto 0) := to_unsigned(SEED, 32);
191     signal bp_lfsr   : unsigned(7 downto 0)  := x"5A";
192     signal err_lfsr  : unsigned(7 downto 0)  := x"C3";
```

Signal	Width / seed	Used for
lfsr	32 bits, SEED	Random frame length (bits 15..0). Taps 31,21,1,0: a standard maximal-length polynomial ($x^{32}+x^{22}+x^2+x+1$).
bp_lfsr	8 bits, 0x5A	TX ready stall pattern.
err_lfsr	8 bits, 0xC3	RX error injection.

Three independent generators are used so the stall pattern and the error pattern do not correlate with the length pattern. The intent is right; the 8-bit polynomial is not (section 11).

5.3 RX generator state (lines 194-202)

```
194     -----
195     -- RX generator state
196     -----
197     signal rx_seq      : unsigned(31 downto 0) := (others => '0');
198     signal rx_len      : natural := MIN_ETH_FRAME;
199     signal rx_off       : natural := 0;
200     signal rx_active   : boolean := false;
201     signal gap_cnt     : natural := 0;
202     signal ipg_cnt     : natural := 0;
```

Signal	Holds between clock cycles
rx_seq	Sequence number of the next frame. Written into bytes 14-17 of every frame.
rx_len	Length of the frame currently being emitted (before stripping).
rx_off	Byte offset reached within that frame.
rx_active	A frame is in progress and must continue in the next beat.
gap_cnt	Idle segments still owed after the last EOP.
ipg_cnt	Intended IPG counter. Never loaded (section 12.8).

This is the minimum state needed to continue a frame across a beat boundary: which frame (seq, len), how far into it (off), whether one is open (active), and how much idle is owed (gap).

5.4 TX checker state and internal counters (lines 204-218)

```
204     -----
205     -- TX protocol checker state
206     -----
207     signal rdy_pipe    : std_logic_vector(7 downto 0) := (others => '0');
```

```

208     signal tx_d_hold  : std_logic_vector(DATA_W-1 downto 0) := (others => '0');
209     signal tx_if_hold : std_logic_vector(NUM_SEG-1 downto 0) := (others => '0');
210     signal have_hold  : boolean := false;
211     signal settle     : natural range 0 to 255 := 0;
212
213     -- VHDL-93: out ports cannot be read, so count internally
214     constant ZER016 : std_logic_vector(NUM_SEG-1 downto 0) := (others => '0');
215     signal cnt_frames : unsigned(31 downto 0) := (others => '0');
216     signal cnt_bad    : unsigned(31 downto 0) := (others => '0');
217     signal cnt_viol   : unsigned(31 downto 0) := (others => '0');
218     signal tx_ready_i : std_logic := '0';

```

Signal	Purpose
rdy_pipe	8-deep shift register of past ready values. Bit L-1 is ready from L cycles ago: the expected valid.
tx_d_hold / tx_if_hold	Last data and inframe seen with valid='1'. Used to check the bus stays frozen while valid='0'.
have_hold	A hold value exists (prevents a false violation before the first valid beat).
settle	Cycles since TX reset; checking starts after 64.
cnt_frames / cnt_bad / cnt_viol	Internal counters behind the o_model_* ports.
tx_ready_i	Internal copy of o_tx_mac_ready so it can be read (shifted into rdy_pipe).

Why internal copies. VHDL-93 forbids reading a port of mode **out**. Quartus parses VHDL-93 by default and rejected earlier versions with error 10309. Counting in an internal signal and assigning the port from it is legal in both VHDL-93 and VHDL-2008 and is the standard pattern. (VHDL-2008 allows reading **out** ports, but keeping to the 93 subset means the file works with either tool default.)

ZER016 is a named all-zeros constant of width **NUM_SEG**, used for the 'beat is empty' comparison on line 496. Its name says 16 but its width follows **NUM_SEG** — fine today, misleading if **NUM_SEG** ever changes.

6. Function frame_byte (lines 220-247)

```
220  -----
221  -- Frame content generator
222  --
223  -- [ 6B DA ][ 6B SA ][ 2B len/type ][ 4B seq ][ payload pattern ]
224  -- The sequence number lets a checker detect drops, duplication and
225  -- reordering independently of payload corruption.
226  -----
227  function frame_byte(idx : natural; sq : unsigned) return std_logic_vector is
228  begin
229      case idx is
230          when 0|1|2|3|4|5 => return x"FF";           -- DA broadcast
231          when 6           => return x"00";           -- SA
232          when 7           => return x"1A";
233          when 8           => return x"2B";
234          when 9           => return x"3C";
235          when 10          => return x"4D";
236          when 11          => return x"5E";
237          when 12          => return x"08";           -- Length/Type hi
238          when 13          => return x"00";           -- Length/Type lo
239          when 14          => return std_logic_vector(sq(31 downto 24));
240          when 15          => return std_logic_vector(sq(23 downto 16));
241          when 16          => return std_logic_vector(sq(15 downto 8));
242          when 17          => return std_logic_vector(sq(7  downto 0));
243          when others      =>
244              return std_logic_vector(
245                  to_unsigned(idx mod 256, 8) xor sq(7 downto 0));
246      end case;
247  end function;
```

What it does. Returns byte number `idx` of the frame with sequence number `sq`. It is a pure function of (`idx`, `sq`), so any checker can regenerate the expected byte without storing frames.

Bytes	Content	Why
0-5	FF FF FF FF FF FF	Broadcast destination MAC. Explains why every frame's status is RXSTAT_BCAST_MC (line 468).
6-11	00 1A 2B 3C 4D 5E	Fixed source MAC. Recognisable in waveforms.
12-13	08 00	EtherType IPv4. A type, not a length (value ≥ 1536), which matters for pad stripping.
14-17	seq[31:0], MSB first	Detects drop, duplication and reordering independently of payload corruption.
18+	(idx mod 256) xor seq[7:0]	Payload differs between frames, so a stuck pipeline replaying an old beat cannot look correct.

VHDL detail. `when 0|1|2|3|4|5 =>` is a choice list; `when others` is mandatory because `idx` is a `natural`, and case statements must cover every value of the selector type.

Weakness. Only 8 bits of the sequence number enter the payload. Two frames 256 apart have identical payloads, so a segment swap between them is invisible in the payload check (the header `seq` still differs, so it is caught only if the checker compares bytes 14-17 of every frame). A hash of (`seq`, `idx`) would make every byte unique.

7. Function stripped_len (lines 249-262)

```
249  -- Effective frame length after 'Bytes to remove from RX frames'
250  function stripped_len(len : natural) return natural is
251  begin
252      case RX_STRIP is
253          when STRIP_NONE    => return len;           -- FCS kept
254          when STRIP_CRC     => return len - 4;       -- FCS removed
255          when STRIP_CRC_PAD => return len - 4;       -- FCS and padding
256          if len <= MIN_ETH_FRAME then
257              return MIN_ETH_FRAME - 4;
258          else
259              return len - 4;
260          end if;
261      end case;
262  end function;
```

What it does. Converts the length on the wire into the length the client actually receives, per the 'Bytes to remove from RX frames' option. The generator then emits only `stripped_len(len)` bytes (line 426).

- **STRIP_NONE**: FCS delivered, length unchanged. The last 4 bytes are pattern, not a real CRC, so any FCS check you add downstream will fail.
- **STRIP_CRC**: 4 bytes removed. A 64 B frame arrives as 60 B = 8 segments with `eop_empty=4`.
- **STRIP_CRC_PAD**: **equivalent to STRIP_CRC for every legal length**. For `len=64` it returns `60 = len-4`; otherwise `len-4`. Real pad stripping depends on the Length/Type field (only when it is a length below 1536, strip to 14 + length). The generator always writes 0x0800, a type, so no pad would be stripped anyway. The branch looks like a feature but does nothing extra.

Why it matters to you. For an encryptor you normally want CRC stripped on RX (you are going to change the frame, so the old FCS is useless) and regenerated by the TX MAC after encapsulation (`skip_crc='0'`).

8. Concurrent assignments (lines 264-269)

```

264 begin
265
266     o_model_rx_frames  <= cnt_frames;
267     o_model_bad_frames <= cnt_bad;
268     o_model_proto_viol <= cnt_viol;
269     o_tx_mac_ready     <= tx_ready_i;

```

The only way internal values reach **out** ports under VHDL-93. These are continuous assignments: each port always equals its internal signal with no clock delay.

9. Process reset_seq (lines 271-306)

```
271  -----
272  -- RESET AND STATUS SEQUENCING
273  -----
274  reset_seq : process(i_clk_rx)
275  begin
276      if rising_edge(i_clk_rx) then
277          if i_rst_n = '0' then
278              rst_cnt    <= 0;
279              s_rst_ack_n <= '0';
280          elsif rst_cnt < RST_ACK_CYC then
281              rst_cnt    <= rst_cnt + 1;
282          else
283              s_rst_ack_n <= '1';          -- IP no longer in full reset
284          end if;
285
286          -- TX datapath
287          if i_rst_n = '0' or i_tx_rst_n = '0' then
288              tx_cnt    <= 0;
289              s_lanes    <= '0';
290          elsif s_rst_ack_n = '1' and tx_cnt < LANE_CYC then
291              tx_cnt    <= tx_cnt + 1;
292          elsif tx_cnt >= LANE_CYC then
293              s_lanes    <= '1';          -- TX lanes stable
294          end if;
295
296          -- RX datapath
297          if i_rst_n = '0' or i_rx_rst_n = '0' then
298              rx_cnt    <= 0;
299              s_pcs_rdy <= '0';
300          elsif s_rst_ack_n = '1' and rx_cnt < PCS_CYC then
301              rx_cnt    <= rx_cnt + 1;
302          elsif rx_cnt >= PCS_CYC then
303              s_pcs_rdy <= '1';          -- RX PCS ready
304          end if;
305      end if;
306  end process;
```

Line by line

- **276:** everything is synchronous to `i_clk_rx`. Legal because in segmented mode TX and RX clocks are the same clock.
- **277-284 (full reset):** while `i_rst_n='0'`, the counter clears and the ack is held low. After release, the counter runs to 8, then `s_rst_ack_n` goes high on the following cycle and stays there.
- **287-294 (TX):** TX is held in reset by either the full or the TX reset. It counts only after the full ack has come up (`s_rst_ack_n='1'`), so TX can never become stable before the IP is out of full reset. After 24 counts `s_lanes='1'`.
- **297-304 (RX):** identical structure with 32 counts, producing `s_pcs_rdy`.
- The `elsif` chain order matters: 'count while below the limit' comes before 'set the flag once at the limit', so the flag rises exactly one cycle after the counter saturates and then holds.

Approximate timeline after i_rst_n is released (tx/rx resets already high)

Cycle	Event
0-8	rst_cnt counts 0 -> 8
~9	s_rst_ack_n = '1' (o_rst_ack_n, o_tx_rst_ack_n, o_rx_rst_ack_n all rise together)
~10-33	tx_cnt counts to 24
~34	s_lanes = '1' -> o_tx_lanes_stable; TX ready generator may now assert ready
~42	s_pcs_rdy = '1' -> PCS ready, locks, alignment; local fault clears; RX traffic may start

Why this ordering

The UG (Table 43, quoted on lines 171-173) says `o_tx_mac_ready` is only meaningful after `o_tx_lanes_stable`. Driving TX earlier is a common reason a link never comes up. The model enforces the ordering by keeping ready low until lanes are stable, so a datapath that ignores the rule shows no TX activity in simulation.

What is wrong

- The model's rise order is fixed: TX lanes always stable before RX PCS ready. A datapath that silently assumes this order passes.
- `o_tx_rst_ack_n` and `o_rx_rst_ack_n` never respond to `i_tx_rst_n/i_rx_rst_n`. Per-direction reset — your recovery path — gets no handshake.
- The acknowledge is effectively instantaneous: `ack_n` is low for as long as reset is low. A user who releases reset before seeing the acknowledge cannot be caught.

10. Status assignments (lines 308-320)

```

308     o_rst_ack_n    <= s_rst_ack_n;
309     -- UG: the exact sequence and timing of the TX/RX acks is NOT guaranteed.
310     o_tx_rst_ack_n <= s_rst_ack_n;
311     o_rx_rst_ack_n <= s_rst_ack_n;
312
313     o_tx_lanes_stable <= s_lanes;
314     o_rx_pcs_ready    <= s_pcs_rdy;
315     o_rx_block_lock   <= s_pcs_rdy;
316     o_rx_am_lock      <= s_pcs_rdy;
317     o_rx_pcs_fully_aligned <= s_pcs_rdy;
318     o_local_fault_status <= not s_pcs_rdy;
319     o_remote_fault_status <= '0';
320     o_rx_hi_ber        <= '0';

```

All three acks come from one signal, even though the comment on line 309 says their order is not guaranteed. That is the opposite of what a model should do: if the order is unspecified, the model should vary it so that code depending on any particular order fails. The four lock/alignment signals collapse to `s_pcs_rdy`, local fault is its inverse, and remote fault and hi-BER are hardwired '0'. The link never goes down during traffic, so the behaviour of a frame caught half-way through your pipeline when PCS drops is untested.

```

-- Suggested: independent ack timing so ordering bugs surface
signal s_tx_ack_n, s_rx_ack_n : std_logic := '0';
...
o_tx_rst_ack_n <= s_tx_ack_n; -- own counter, e.g. RST_ACK_CYC + 3,
o_rx_rst_ack_n <= s_rx_ack_n; -- own counter, e.g. RST_ACK_CYC + 11,
-- each also driven low by its own i*_rst_n

```

11. Process rng (lines 322-338)

```

322     -----
323     -- LFSRs
324     -----
325     rng : process(i_clk_rx)
326     begin
327         if rising_edge(i_clk_rx) then
328             if i_rst_n = '0' then
329                 lfsr    <= to_unsigned(SEED, 32);
330                 err_lfsr <= x"C3";
331             else
332                 lfsr    <= lfsr(30 downto 0) &
333                     (lfsr(31) xor lfsr(21) xor lfsr(1) xor lfsr(0));
334                 err_lfsr <= err_lfsr(6 downto 0) &
335                     (err_lfsr(7) xor err_lfsr(5));
336             end if;
337         end if;
338     end process;

```

How an LFSR works here. Each clock the register shifts left by one bit, and the new bit 0 is the XOR of selected 'tap' bits. With the right taps the register visits every non-zero state before repeating (maximal length, period $2^n - 1$).

- 32-bit `lfsr`, taps 31,21,1,0: maximal length (period $2^{32} - 1$). Correct.
- 8-bit `err_lfsr`, taps 7,5: **period 30, not 255**. I simulated it: starting from 0xC3 it cycles through 30 values and repeats.

Setting	Intended	Actual (period-30 LFSR)
ERR_RATE = 64	~25% (64/256)	10 of 30 states below 64 = 33%
ERR_RATE = 255	all frames	29 of 30 = 97%
STALL_RATE = 40 (bp_lfsr, seed 0x5A)	~16%	3 of 30 = 10%, fixed 30-cycle pattern

A maximal 8-bit polynomial is $x^8+x^6+x^5+x^4+1$, i.e. taps 7,5,4,3:

```
err_lfsr <= err_lfsr(6 downto 0) &  
    (err_lfsr(7) xor err_lfsr(5) xor err_lfsr(4) xor err_lfsr(3));
```

Note `bp_lfsr` is advanced in `tx_ready_gen`, not here, so the TX stall source is reset by the TX reset and `err_lfsr` by the full reset.

12. Process rx_gen — the RX traffic generator (lines 340-517)

This is the core of the model. Every clock it builds one complete 128-byte beat by walking the 16 segments in order, continuing a frame from the previous beat if one is open, ending it, inserting gap segments and starting new frames — possibly several of each in one beat.

12.1 Header comment and variable declarations (lines 340-367)

```
340  -----
341  -- RX TRAFFIC GENERATOR
342  --
343  -- Frames are packed TIGHTLY by default (i_gap_segs = 0), which is the
344  -- maximum-throughput case UG sec 7.4 mandates for TX. Setting i_gap_segs
345  -- >= 1 produces beats where inframe has HOLES, matching the loosely
346  -- packed case drawn in the UG RX waveform. Both are legal RX streams and
347  -- a datapath must survive either.
348  -----
349  rx_gen : process(i_clk_rx)
350      variable v_data : std_logic_vector(DATA_W-1 downto 0);
351      variable v_if   : std_logic_vector(NUM_SEG-1 downto 0);
352      variable v_empty : std_logic_vector(NUM_SEG*EMPTY_W-1 downto 0);
353      variable v_fcs   : std_logic_vector(NUM_SEG-1 downto 0);
354      variable v_err   : std_logic_vector(NUM_SEG*ERR_W-1 downto 0);
355      variable v_stat  : std_logic_vector(NUM_SEG*STAT_W-1 downto 0);
356      variable v_len   : natural;
357      variable v_off   : natural;
358      variable v_seq   : unsigned(31 downto 0);
359      variable v_act   : boolean;
360      variable v_gap   : natural;
361      variable v_nf    : natural;
362      variable v_nbad  : natural;
363      variable v_eff   : natural;
364      variable bad     : boolean;
365      variable oversz  : boolean;
366      variable b       : natural;
367  begin
```

Why variables. A VHDL signal assigned inside a process only takes its new value when the process suspends. Within one clock the generator must update frame state many times (a frame can end in segment 7 and the next start in segment 9). Variables update immediately, so the loop can carry state from segment to segment. At the end of the beat the variables are copied back into signals (lines 482-510), which is what makes the state survive into the next clock.

The `v_*` variables are working copies of: the beat's output vectors (`v_data`, `v_if`, `v_empty`, `v_fcs`, `v_err`, `v_stat`), the frame state (`v_len`, `v_off`, `v_seq`, `v_act`, `v_gap`), per-beat counts (`v_nf` frames finished, `v_nbad` bad frames), and scratch values (`v_eff`, `bad`, `oversz`, `b`).

12.2 Reset branch (lines 368-385)

```
368      if rising_edge(i_clk_rx) then
369          if i_rst_n = '0' or i_rx_rst_n = '0' then
370              o_rx_mac_data    <= (others => '0');
371              o_rx_mac_valid   <= '0';
372              o_rx_mac_inframe <= (others => '0');
373              o_rx_mac_eop_empty <= (others => '0');
374              o_rx_mac_fcs_error <= (others => '0');
375              o_rx_mac_error   <= (others => '0');
376              o_rx_mac_status_data <= (others => '0');
377              rx_seq          <= (others => '0');
378              rx_len          <= MIN_ETH_FRAME;
379              rx_off          <= 0;
380              rx_active       <= false;
381              gap_cnt         <= 0;
382              ipg_cnt         <= 0;
383              cnt_frames      <= (others => '0');
384              cnt_bad         <= (others => '0');
385          end if;
```

Either full reset or RX reset clears all outputs and all generator state. Clearing the counters here means each reset starts a fresh count, so the testbench must sample totals before any mid-run reset.

12.3 Run condition and variable initialisation (lines 386-402)

```
386      -- Traffic only after the RX datapath reports ready.
```

```

387         elsif s_pcs_rdy = '1' and (i_gen_enable = '1' or rx_active) then
388
389             v_data := (others => '0');
390             v_if    := (others => '0');
391             v_empty := (others => '0');
392             v_fcs   := (others => '0');
393             v_err   := (others => '0');
394             v_stat  := (others => '0');
395             v_len   := rx_len;
396             v_off   := rx_off;
397             v_seq   := rx_seq;
398             v_act   := rx_active;
399             v_gap   := gap_cnt;
400             v_nf    := 0;
401             v_nbad  := 0;
402

```

- **387**: traffic only after PCS ready. The condition `i_gen_enable = '1' or rx_active` means that once a frame has started it always finishes, even if the testbench drops `i_gen_enable` mid-frame. That guarantees the generator never produces a truncated frame, which keeps the scoreboard simple.
- **389-394**: all per-beat outputs start at zero; a segment nobody writes stays idle (inframe='0', data 0).
- **395-399**: load working copies of the frame state from the signals.
- **400-401**: per-beat counters start at zero.

12.4 Segment loop: gap handling and frame start (lines 403-422)

```

403         for s in 0 to NUM_SEG-1 loop
404             if v_gap /= 0 then
405                 -- idle segment between frames: inframe stays '0',
406                 -- leaving a hole in the beat
407                 v_gap := v_gap - 1;
408             else
409                 if not v_act then
410                     if i_gen_enable = '1' then
411                         if i_force_len /= 0 then
412                             v_len := to_integer(i_force_len);
413                         else
414                             v_len := MIN_LEN +
415                                 (to_integer(lfsr(15 downto 0))
416                                  mod (MAX_LEN - MIN_LEN + 1));
417                         end if;
418                         v_off := 0;
419                         v_act := true;
420                     end if;
421                 end if;
422

```

- **404-407**: if gap segments are still owed, this segment stays idle (inframe '0') and the gap count decrements. This is what creates the 'holes' in inframe that make EOP visible.
- **409-420**: no frame open and generation enabled -> start one. Length is either forced or `MIN_LEN + (lfsr[15:0] mod (range))`. Offset resets to 0; the frame is now active.

Finding 1 (part A): `lfsr` is a signal, so it has one value for the whole loop. If two frames start in the same beat they get the same random length. The same applies to the error decision below.

Minor: the modulo of a 16-bit value by a range that does not divide 65536 has a slight bias towards smaller lengths. Harmless for stimulus.

12.5 Segment loop: filling data (lines 423-437)

```

423         if v_act then
424             -- 'Bytes to remove from RX frames' applies to what
425             -- the client sees, not to what was on the wire.
426             v_eff := stripped_len(v_len);
427
428             for k in 0 to SEG_B-1 loop
429                 b := v_off + k;
430                 if b < v_eff then
431                     v_data((s*SEG_B+k)*8+7 downto (s*SEG_B+k)*8)
432                         := frame_byte(b, v_seq);
433                 end if;
434             end loop;
435
436             v_if(s) := '1';
437

```

- **426**: compute the length the client will actually see (after CRC/pad stripping).
- **428-434**: for each of the 8 byte lanes in this segment, take frame byte `v_off + k` if it is still inside the frame; bytes past the end stay 0. The slice `(s*SEG_B+k)*8+7 downto (s*SEG_B+k)*8` places segment 0 byte 0 at bits 7..0.
- **436**: mark the segment as inside a frame.

12.6 Segment loop: end of packet, errors and status (lines 438-478)

```

438         if (v_off + SEG_B) >= v_eff then
439             -- EOP in this segment
440             v_empty(s*EMPTY_W+EMPTY_W-1 downto s*EMPTY_W)
441                 := std_logic_vector(to_unsigned(
442                     SEG_B - (v_eff - v_off), EMPTY_W));
443
444             -- Error injection. UG Table 46: error and
445             -- fcs_error are valid on EOP segments.
446             bad := (i_err_rate /= 0) and
447                 (err_lfsr < i_err_rate);
448
449             -- 'Enforce maximum frame size': the IP
450             -- truncates oversized packets and the error
451             -- signal indicates oversize AND FCS error.
452             oversz := ENFORCE_MAX and (v_len > RX_MAX_FRAME);
453
454             if oversz then
455                 v_fcs(s) := '1';
456                 v_err(s*ERR_W+ERR_W-1 downto s*ERR_W)
457                     := RXERR_SIZE;
458                 v_nbad := v_nbad + 1;
459             elsif bad then
460                 v_fcs(s) := '1';
461                 v_err(s*ERR_W+ERR_W-1 downto s*ERR_W)
462                     := RXERR_MALFORM;
463                 v_nbad := v_nbad + 1;
464             end if;
465
466             -- Status: broadcast DA in this generator
467             v_stat(s*STAT_W+STAT_W-1 downto s*STAT_W)
468                 := RXSTAT_BCAST_MC;
469
470             v_seq := v_seq + 1;
471             v_nf := v_nf + 1;
472             v_act := false;
473             v_off := 0;
474             v_gap := to_integer(i_gap_segs);
475         else
476             v_off := v_off + SEG_B;
477         end if;
478     end if;

```

- **438**: EOP if the remaining bytes (`v_eff - v_off`) fit in this segment.
- **440-442**: `eop_empty` = 8 minus the bytes used. For a 60 B frame the last segment carries bytes 56-59, so `empty` = 4.
- **446-447**: error decision — random if `i_err_rate` is non-zero.
- **452**: oversize only when `ENFORCE_MAX` is set and the wire length exceeds `RX_MAX_FRAME`.
- **454-464**: oversize takes priority (`fcs_error + RXERR_SIZE`); otherwise a random 'bad' frame gets `fcs_error + RXERR_MALFORM`. Errors are written on the EOP segment only, which matches the UG.
- **467-468**: status on the EOP segment: always broadcast/multicast, because the DA is always FF..FF.
- **470-474**: frame done: next sequence number, count it, close it, reset offset, and load the gap counter for the idle segments that follow.
- **476**: not EOP -> advance 8 bytes and continue in the next segment.

Finding 1 (part B): `err_lfsr` is a signal, so `bad` is identical for every EOP in the same beat. With 64 B frames and `GAP_SEGS=1` (9 segments per frame), two EOPs fall into one beat regularly — and are always both good or both bad. The case the drop stage must handle, a good and a bad frame in the same beat, is never produced.

Finding 7: `ENFORCE_MAX` flags the frame but still emits all `v_eff` bytes — the frame is not truncated to `RX_MAX_FRAME` as the comment on line 449 says. With `ENFORCE_MAX=false` an oversize frame is not

flagged at all. Frames under 64 B (reachable via `i_force_len`) are never flagged `RXERR_SIZE`. VLAN/SVLAN/PFC status codes are never generated, although you need VLAN/QinQ/MPLS support.

12.7 Writing the beat out (lines 479-487)

```

479             end if;
480         end loop;
481
482         o_rx_mac_data      <= v_data;
483         o_rx_mac_inframe   <= v_if;
484         o_rx_mac_eop_empty <= v_empty;
485         o_rx_mac_fcs_error <= v_fcs;
486         o_rx_mac_error     <= v_err;
487         o_rx_mac_status_data <= v_stat;

```

After the loop, the fully built beat is assigned to the output ports in one go. Because these are signal assignments on a clock edge, the outputs are registered: the beat appears one cycle after it is computed.

12.8 valid and the IPG counter (lines 489-501)

```

489         -- 'Average inter-packet gap': below 12 the IP no longer
490         -- complies with the Ethernet standard but throughput rises,
491         -- most noticeably at small frame sizes.
492         if ipg_cnt /= 0 then
493             o_rx_mac_valid <= '0';
494             ipg_cnt        <= ipg_cnt - 1;
495         else
496             if v_if = ZERO16 then
497                 o_rx_mac_valid <= '0';
498             else
499                 o_rx_mac_valid <= '1';
500             end if;
501         end if;

```

Intent. Insert idle cycles to model the inter-packet gap setting. **Reality.** `ipg_cnt` is set to 0 in reset and only ever decremented from a non-zero value — it is never loaded with anything — so the first branch is unreachable and `AVG_IPG` has no effect.

Even if it were loaded, forcing `valid='0'` here would be wrong: the beat's data has already been generated and the frame state has already advanced, so dropping `valid` would lose that data. IPG belongs in the gap, as idle segments:

```

-- at EOP, replacing line 474 (approximation of client-side spacing)
v_gap := maximum(to_integer(i_gap_segs),
                 (AVG_IPG + 8 + SEG_B - 1) / SEG_B); -- IPG + preamble/SFD

```

At `AVG_IPG=12` this gives 3 idle segments per frame. For 64 B frames that is 11 segments per frame, about 1.45 frames/beat, about 604 Mpps — close to the ~595 Mpps physical 400GE maximum. It also makes a zero gap impossible by construction.

valid='0' on empty beats (496-498). A beat with no inframe bits carries no data, so `valid` is dropped. Edge case: a frame whose last segment is 15 is only terminated by the next beat's `inframe(0)='0'`. If that next beat is empty and marked invalid, a consumer that only looks at valid beats keeps the frame open until the next frame arrives — after the final frame of a burst, possibly indefinitely. Check that your drop stage handles an EOP in segment 15, and check whether the real IP marks idle RX beats invalid at all.

12.9 State write-back and idle branch (lines 503-517)

```

503         rx_len      <= v_len;
504         rx_off       <= v_off;
505         rx_seq       <= v_seq;
506         rx_active    <= v_act;
507         gap_cnt      <= v_gap;
508
509         cnt_frames   <= cnt_frames + v_nf;
510         cnt_bad      <= cnt_bad    + v_nbad;
511
512         else
513             o_rx_mac_valid <= '0';
514             o_rx_mac_inframe <= (others => '0');
515         end if;
516     end if;

```

517 end process;

The working variables go back into the state signals so the next beat continues where this one stopped. The frame counters add the number of frames finished in this beat, which can be 0, 1, 2 or more. The **else** branch (PCS not ready, or generation stopped with no frame open) outputs idle. It leaves data, eop_empty and error at their last values — harmless because inframe and valid are zero.

12.10 Worked example: 64 B frames, STRIP_CRC, GAP_SEGS=1

Each wire frame is 64 B, delivered as 60 B = 8 segments (7 full + one with 4 bytes, eop_empty=4), followed by 1 gap segment: a 9-segment pattern. A beat is 16 segments.

Beat	Segments 0..15 (F=frame n, g=gap, * = EOP)	inframe (seg15..seg0)	EOPs
0	F0 F0 F0 F0 F0 F0 F0 F0* g F1 F1 F1 F1 F1 F1 F1	0xFEFF	1
1	F1* g F2 F2 F2 F2 F2 F2 F2* g F3 F3 F3 F3 F3	0xFBFD	2 (F1, F2)
2	F3 F3 F3* g F4 F4 F4 F4 F4 F4 F4* g F5 F5 F5	0xEFF7	2 (F3, F4)

$16/9 = 1.778$ frames per beat, matching the README. Beats 1 and 2 each carry two EOPs — and because of finding 1, those two frames always receive the same error verdict.

With GAP_SEGS=0 there are no g segments: inframe = 0xFFFF on every beat, and no 1->0 transition exists between F0 and F1. By the definition on line 34 that boundary does not exist. This is why finding 4 calls the gap=0 stream unframeable rather than a harder stress case.

13. Process tx_ready_gen (lines 519-550)

```
519  -----
520  -- TX READY GENERATOR
521  --
522  -- o_tx_mac_ready IS a real IP output (UG Table 43) and the pause protocol
523  -- it drives IS documented (UG sec 7.4). But the RATE and PATTERN of
524  -- stalling are NOT specified anywhere in the UG. The LFSR below is an
525  -- arbitrary stress pattern - useful for proving the datapath survives
526  -- frequent stalls, NOT a prediction of real IP behaviour.
527  --
528  -- Ready is gated on o_tx_lanes_stable, per UG Table 43.
529  -----
530  tx_ready_gen : process(i_clk_tx)
531  begin
532      if rising_edge(i_clk_tx) then
533          if i_rst_n = '0' or i_tx_rst_n = '0' then
534              bp_lfsr      <= x"5A";
535              tx_ready_i <= '0';
536          else
537              bp_lfsr <= bp_lfsr(6 downto 0) &
538                          (bp_lfsr(7) xor bp_lfsr(5));
539              if s_lanes = '0' then
540                  tx_ready_i <= '0';
541              elsif i_tx_stall_rate = 0 then
542                  tx_ready_i <= '1';
543              elsif bp_lfsr > i_tx_stall_rate then
544                  tx_ready_i <= '1';
545              else
546                  tx_ready_i <= '0';
547              end if;
548          end if;
549      end if;
550  end process;
```

- **533-535:** TX reset clears the stall LFSR and ready.
- **537-538:** advance the stall LFSR every cycle (same period-30 polynomial as err_lfsr).
- **539-540:** ready stays low until TX lanes are stable — the UG Table 43 rule.
- **541-547:** no stalls if the rate is 0; otherwise ready is high when the LFSR value exceeds the threshold.

Why the comment is careful. The UG documents the pause protocol that ready drives, but not how often or in what pattern the real IP deasserts ready. The comment says so plainly: the LFSR pattern is a stress test, not a prediction. That honesty is correct.

Weakness. Stalls are single-cycle and periodic every 30 cycles. Long bursts of ready='0' — the case that fills your FIFO and triggers overflow — are never generated. A burst mode (hold ready low for N cycles) would cover it.

14. Process tx_check — the TX protocol checker (lines 552-605)

```
552  -----
553  -- TX PROTOCOL CHECKER
554  --
555  -- Enforces UG sec 7.4:
556  -- valid must equal ready delayed by exactly READY_LATENCY, and the bus
557  -- must freeze while valid is deasserted.
558  --
559  -- A model that only supplies stimulus lets protocol bugs through
560  -- silently. This one catches them.
561  -----
562  tx_check : process(i_clk_tx)
563      variable exp_valid : std_logic;
564  begin
565      if rising_edge(i_clk_tx) then
566          if i_rst_n = '0' or i_tx_rst_n = '0' then
567              rdy_pipe <= (others => '0');
568              have_hold <= false;
569              settle <= 0;
570              cnt_viol <= (others => '0');
571          else
572              rdy_pipe <= rdy_pipe(6 downto 0) & tx_ready_i;
573              if settle < 255 then
```

```

574         settle <= settle + 1;
575     end if;
576
577     exp_valid := rdy_pipe(READY_LATENCY-1);
578
579     -- Do not police across reset or before lanes are stable.
580     if settle > 64 and s_lanes = '1' then
581         if i_tx_mac_valid /= exp_valid then
582             cnt_viol <= cnt_viol + 1;
583             report "TX PROTOCOL: valid /= ready delayed by " &
584                 integer'image(READY_LATENCY)
585                 severity warning;
586         end if;
587
588         if i_tx_mac_valid = '0' and have_hold then
589             if i_tx_mac_data /= tx_d_hold or
590                i_tx_mac_inframe /= tx_if_hold then
591                 cnt_viol <= cnt_viol + 1;
592                 report "TX PROTOCOL: bus changed while valid low"
593                     severity warning;
594             end if;
595         end if;
596     end if;
597
598     if i_tx_mac_valid = '1' then
599         tx_d_hold <= i_tx_mac_data;
600         tx_if_hold <= i_tx_mac_inframe;
601         have_hold <= true;
602     end if;
603 end if;
604 end if;
605 end process;

```

How the latency check works

`rdy_pipe` is a shift register: each cycle the current ready enters at bit 0 and everything moves up one place. So at any cycle, bit L-1 holds the ready value from L cycles earlier. The UG rule is that valid now must equal ready L cycles ago, so the checker compares `i_tx_mac_valid` against `rdy_pipe(READY_LATENCY-1)`.

Cycle	tx_ready_i	rdy_pipe(0)	rdy_pipe(1)	rdy_pipe(2) = expected valid (L=3)
t	0	-	-	-
t+1	1	0	-	-
t+2	1	1	0	-
t+3	1	1	1	0 (ready at t)
t+4	1	1	1	1 (ready at t+1)

The check is two-sided: valid high when not expected is a violation, and so is valid low when expected. The second half enforces the UG rule that valid must assert when ready permits it 'even though there is no packet to send' — an idle TX beat is inframe=0 with valid=1, not valid=0.

Line by line

- **572:** shift the ready history.
- **573-575:** count up to 255 after reset.
- **580:** police only after 64 settled cycles and after lanes are stable, so start-up does not produce false violations.
- **581-586:** latency rule. `report ... severity warning` prints to the simulator log without stopping simulation, so one run collects every violation.
- **588-595:** freeze rule: while valid is low, data and inframe must equal the last values seen with valid high.
- **598-602:** capture the bus on every valid beat for the next freeze comparison.

What is wrong

- The freeze check covers only data and inframe. The UG rule covers every TX signal, so eop_empty, error and skip_crc changing during a pause are not caught. Fix:

```

signal tx_e_hold : std_logic_vector(NUM_SEG*EMPTY_W-1 downto 0);
signal tx_er_hold : std_logic_vector(NUM_SEG-1 downto 0);
signal tx_sc_hold : std_logic_vector(NUM_SEG-1 downto 0);

```

```

...
if i_tx_mac_data /= tx_d_hold or i_tx_mac_inframe /= tx_if_hold or
   i_tx_mac_eop_empty /= tx_e_hold or i_tx_mac_error /= tx_er_hold or
   i_tx_mac_skip_crc /= tx_sc_hold then

```

- It checks timing only, never content. Nothing verifies that each EOP is followed by an inframe='0' segment, that eop_empty is zero on non-EOP segments, or that a TX frame is no longer than TX_MAX_FRAME. For an encryptor adding 64 B per frame, a length check is the one you will need first:

```

-- sketch: count bytes of the open TX frame on valid beats
-- for each segment s with inframe(s)='1': len := len + (8 - empty_at_eop)
-- at EOP: if len > TX_MAX_FRAME then cnt_viol++ ; report "TX oversize"

```

- The 64-cycle blind window restarts on every TX reset, so behaviour immediately after a recovery reset is never checked.

README wording to check: it says UG sec 7.4 mandates TX tight packing 'with no idle segments'. Under the inframe definition on line 34 that is impossible — every frame end needs a following inframe='0' segment. 'Tight packing' more plausibly means no idle segments inside a frame and no gap beyond the minimum. Read the exact 7.4 text; it decides how you must pack encapsulated frames on TX.

15. Fixes in priority order

Fix 1 — per-EOP error and length draws with a maximal LFSR

Advance a variable copy of the LFSR at each frame start and each EOP, then write it back once. Remove the corresponding assignments from the `rng` process to avoid two drivers.

```
variable v_elfsr : unsigned(7 downto 0);
variable v_llfsr : unsigned(31 downto 0);
...
v_elfsr := err_lfsr;          -- before the segment loop
v_llfsr := lfsr;
...
-- at frame start (replaces line 414-416):
v_llfsr := v_llfsr(30 downto 0) &
           (v_llfsr(31) xor v_llfsr(21) xor v_llfsr(1) xor v_llfsr(0));
v_len    := MIN_LEN + (to_integer(v_llfsr(15 downto 0))
                      mod (MAX_LEN - MIN_LEN + 1));
...
-- at each EOP (replaces line 446-447):
v_elfsr := v_elfsr(6 downto 0) &
           (v_elfsr(7) xor v_elfsr(5) xor v_elfsr(4) xor v_elfsr(3));
bad := (i_err_rate /= 0) and (v_elfsr < i_err_rate);
...
err_lfsr <= v_elfsr;          -- after the loop
lfsr      <= v_llfsr;
```

Fix 2 — IPG as idle segments; forbid zero gap

Use the snippet in 12.8 and delete the `ipg_cnt` logic. Expect the throughput numbers in the README to fall towards the real 400GE line rate — that is the model becoming correct, not the datapath getting worse.

Fix 3 — reset and link realism

- Separate TX and RX ack counters with different delays, each also driven by its own per-direction reset.
- Hold `ack_n` low for a few cycles after reset is applied, and flag a violation if the user releases reset before seeing it.
- Add a model-control input that drops PCS ready (and raises local fault / hi-BER) mid-traffic, truncating the frame in progress with an error.

Fix 4 — complete TX checking

- Extend the freeze check to `eop_empty`, error and `skip_crc` (section 14).
- Decode TX frames: EOP followed by an idle segment, `eop_empty` only at EOP, length within `TX_MAX_FRAME`.

Fix 5 — option fidelity

- `ENFORCE_MAX`: stop emitting bytes at `RX_MAX_FRAME` and flag the truncated EOP.
- Flag oversize (per the UG rules for your IP version) and undersize regardless of `ENFORCE_MAX`.
- Either implement `STRIP_CRC_PAD` properly (length field below 1536) or delete it.
- Add VLAN / QinQ / MPLS frames to the generator so status codes other than `BCAST_MC` appear.

After fixes 1 and 2, rerun the README's result table. If the drop counts are still exact with mixed good/bad frames in the same beat and realistic spacing, that result means considerably more than the current one.

Appendix — package constants used by this file

Constant	Value	Meaning
DATA_W	1024	Client data bus width, bits
NUM_SEG	16	Segments per beat
SEG_B / SEG_W	8 / 64	Bytes / bits per segment

Constant	Value	Meaning
EMPTY_W / ERR_W / STAT_W	3 / 2 / 3	Per-segment widths of eop_empty, rx error, rx status
BEAT_B	128	Bytes per beat
CLK_PERIOD_PS	2409.5 ps	Testbench clock period (~415.02 MHz; exact 415.0390625 MHz is 2409.41 ps — negligible)
MIN_ETH_FRAME / DEF_MAX_FRAME / JUMBO_MAX	64 / 1518 / 9000	Frame limits
RXERR_*	00 none, 01 malformed, 10 size, 11 payload length	o_rx_mac_error encoding
RXSTAT_*	000..111	o_rx_mac_status_data encoding; generator only uses 010 (bcast/mcast)
TUNNEL_OVH_B / _SEG	64 / 8	IP 20 + UDP 8 + IV 16 + hash 20; exactly 8 segments, so header insertion is segment-aligned rewiring